Introduction and Learning Paradigms

Contents

1.1 Setting expectations **2**

1.2 Learning Paradigms **2**
1.2.1 Supervised learning 3
1.2.2 Unsupervised learning 4
1.2.3 Reinforcement learning 5
1.2.4 Summarized overview 7

1.3 Connectionist Neuron (Perceptron) **8**
1.3.1 Components of the connectionist neuron 8
1.3.2 Linear vs. non-linear transfer functions 10
1.3.3 Shortcut notation for weights and bias 12

1.4 References **12**

1.1 Setting expectations

breadth of topics, not necessarily state of the art theoretical aspects of the learning algorithms

implementing the algorithms - programming exercises.

1.2 Learning Paradigms

There are three learning paradigms in machine learning:

- Supervised learning. Learning with a teacher (cf. Section 1.2.1).
- Unsupervised learning. Learning only from observations, without a teacher (cf. Section 1.2.2).
- Reinforcement learning. This is about maximizing reward (cf. Section 1.2.3).
- Evolutionary learning. Not covered.

It is possible to understand the differences between them by describing (i) the data each of them deals with and (ii) the objective that each is trying to model.

1.2.1 Supervised learning

We are given data that is made up of tuples.

Supervised methods try to fit a function that maps $\underline{\mathbf{x}}$ to some attribute $\underline{\mathbf{y}}$ (map observations to labels/ground truth).

example image recognition

Supervised learning would fit a function that maps image pixels to recognizing the object in the image from a set of possible objects (e.g. cat, dog, tiger, duck, frog, fish).

Supervised learning is essentially function fitting.

Data:

Consider this dataset of tuples:

$$\left(\underline{\mathbf{x}}^{(1)}, \underline{\mathbf{y}}_T^{(1)}\right) \ , \ \ldots \ , \ \left(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{y}}_T^{(\alpha)}\right) \ , \ \ldots \ , \ \left(\underline{\mathbf{x}}^{(p)}, \underline{\mathbf{y}}_T^{(p)}\right) \tag{1.2.1}$$

where,

$\underline{\mathbf{x}}$ denotes the observation with N components. $\underline{\mathbf{x}} = (x_1, x_2, \ldots x_N)$, $\underline{\mathbf{x}} \in \mathbb{R}^N$ (e.g. pixel values in an image)

$\underline{\mathbf{y}}_T$ denotes the label assigned to this observation. It is also referred to as the ground truth. When $y_T \in \mathbb{R}$ or $\underline{\mathbf{y}}_T \in \mathbb{R}^M$, we are looking at a regression problem (e.g. predicting house prices, object localization - bounding box regression (x, y, w, h)). When $y_T \in \{0, 1\}$, we refer to this as a binary classification problem (e.g. is this a cat or a dog). When $y_T \in \{0, \ldots, K-1\}$, we refer to this as a multi-class classification problem (e.g. which letter is this?).

p denotes the size of the dataset (i.e. how many labeled observations we have.)

The superscript $^{(\alpha)}$ denotes the index of a particular sample within the dataset. $\alpha = 1, \ldots, p$

The samples are often assumed to be *independent and identically distributed* (i.i.d.).

Objective model:

Supervised learning algorithms are used to find a function that maps observations to their label(s) which represents some concept or value. This mapping can be described in the form of:

- a discriminative function $\underline{\mathbf{y}} = \underline{\mathbf{f}}(\underline{\mathbf{x}})$,

  where $\underline{\mathbf{f}}(\cdot)$ denotes a vector valued function $\underline{\mathbf{f}} : \mathbb{R}^N \to \mathbb{R}^M$
- a conditional distribution $P(\,\underline{\mathbf{y}}\,|\,\underline{\mathbf{x}}\,)$.

1.2.2 Unsupervised learning

Unsupervised learning tries to find interesting directions and/or structure in the data using only observations $\underline{\mathbf{x}} \in \mathbb{R}^N$.

Data:

A dataset of observations:

$$\underline{\mathbf{X}} = \begin{pmatrix} | & | & & | & & | \\ \underline{\mathbf{x}}^{(1)} & \underline{\mathbf{x}}^{(2)} & \cdots & \underline{\mathbf{x}}^{(\alpha)} & \cdots & \underline{\mathbf{x}}^{(p)} \\ | & | & & | & & | \end{pmatrix} \in \mathbb{R}^{N \times p} \tag{1.2.2}$$

where p denotes the number of observations (i.e. size of the dataset) and N denotes the number of dimensions. The samples are often assumed to be i.i.d., but algorithms for handling sequential data also exist.

Example: $\underline{\mathbf{x}}$ could represent user ratings, pixel values in images.

Objective model:

An unsupervised learning algorithm is used to capture structure or directions in the data. This can be achieved by finding:

- the underlying distribution $P(\underline{\mathbf{x}})$ that generated this data (e.g. density estimation),
- $\underline{\mathbf{z}} := \underline{\mathbf{f}}(\underline{\mathbf{x}})$, where $\underline{\mathbf{z}}$ is a measure of
  - possible structure such as clustering or grouping in the data ($\underline{\mathbf{z}} \in 0, \ldots, K-1$),

    and/or
  - possible directions in the data, by finding another continuous space for describing this data.

    Example: dimensionality reduction, $\underline{\mathbf{z}} \in \mathbb{R}^M$ with $M < N$.

1.2.3 Reinforcement learning

Reinforcement learning is about

what actions to take

in which situations

in order to maximize some reward.

This implies that learning happens through an interaction of the agent with its environment.

Data:

Our data consists of a sequence. Each time step is described by

- a state $\underline{\mathbf{x}} \in \mathcal{X}$ or $\underline{\mathbf{x}} \in \mathbb{R}^N$, e.g. $\mathcal{X} := \{\underline{\mathbf{x}}_1, \ldots, \underline{\mathbf{x}}_S\} \subset \{0,1\}^S$ (1-out-S encoding)
- an action $\underline{\mathbf{a}}$ which can be taken by the agent:

  $\underline{\mathbf{a}} \in \mathcal{A}$ or $\underline{\mathbf{a}} \in \mathbb{R}^M$, e.g. $\mathcal{A} := \{\underline{\mathbf{a}}_1, \ldots, \underline{\mathbf{a}}_A\} \subset \{0,1\}^A$ (1-out-A encoding)
- a reward $r \in \mathbb{R}$ or $r \in \{0,1\}$, e.g. $r \in \{$"cheese", "no cheese"$\}$.

Each time step describes what reward was received when performing some action while in some state. The sequence we observe becomes:

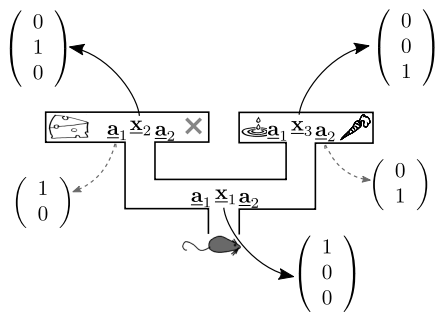
$$\left\{\underline{\mathbf{x}}^{(t)}, \underline{\mathbf{a}}^{(t)}, r^{(t)}\right\}_{t=0}^{p} = \left(\underline{\mathbf{x}}^{(0)}, \underline{\mathbf{a}}^{(0)}, r^{(0)}\right), \ldots, \left(\underline{\mathbf{x}}^{(p)}, \underline{\mathbf{a}}^{(p)}, r^{(p)}\right) \tag{1.2.3}$$

Q 1.2.1: *Can we still assume i.i.d.?* -Since we observe sequential data, we

no longer assume i.i.d. *within* a sequence. But if we have multiple sequences, we can assume i.i.d. *across* sequences.

Example:

- $\underline{\mathbf{x}} \mathrel{\widehat{=}}$ position in a maze
- $\underline{\mathbf{a}} \mathrel{\widehat{=}}$ movement direction (velocity), e.g. turn left/right
- $r \mathrel{\widehat{=}}$ found the cheese, found smaller piece of cheese. A negative reward, "punishment" is also possible.



reinforcement learning example

Q 1.2.2: *What would be a state/action/reward here?*

Objective model:

Reinforcement learning can be used to:

Figure 1.2.1: Representing games for reinforcement learning

- select the optimal action when arriving at a state: $\underline{\mathbf{a}}^* = \underline{\mathbf{f}}(\underline{\mathbf{x}}, r)$
- find the transition model $P(\underline{\mathbf{x}}^{(t)} \mid \underline{\mathbf{x}}^{(t-1)}, \underline{\mathbf{a}}^{(t-1)})$, where t is a time step within the sequence.

Q 1.2.3: *Would you regard reinforcement learning as supervised or unsupervised learning?*

1.2.4 Summarized overview

	Data	Objective model
Supervised learning	$\left(\underline{\mathbf{x}}^{(1)}, \underline{\mathbf{y}}_T^{(1)}\right), \ldots, \left(\underline{\mathbf{x}}^{(p)}, \underline{\mathbf{y}}_T^{(p)}\right)$ ↳ label $\mathbb{R}$, $\mathbb{R}^M$,$\{0,1\}$, $\{0,\ldots,K-1\}$ ↳ observation $\mathbb{R}^N$, often iid.	discrimination function $\underline{\mathbf{y}} = \underline{\mathbf{f}}(\underline{\mathbf{x}})$ conditional distribution $P(\underline{\mathbf{y}}\|\underline{\mathbf{x}})$
Unsupervised learning	$\underline{\mathbf{x}}^{(1)}, \ldots, \underline{\mathbf{x}}^{(p)} \in \mathbb{R}^N$ observations often iid.	generative model/ data distribution $P(\underline{\mathbf{x}})$ clustering: $f(\underline{\mathbf{x}}) : \mathbb{R}^N \mapsto \{0,\ldots,K-1\}$ dimensionality reduction: $\underline{\mathbf{f}}(\underline{\mathbf{x}}) : \mathbb{R}^N \mapsto \mathbb{R}^M$
Reinforcement learning	$\left(\underline{\mathbf{x}}^{(0)}, \underline{\mathbf{a}}^{(0)}, r^{(0)}\right), \ldots, \left(\underline{\mathbf{x}}^{(p)}, \underline{\mathbf{a}}^{(p)}, r^{(p)}\right)$ ↳ reward $r \in \mathbb{R}$ ↳ action $\underline{\mathbf{a}} \in \mathbb{R}^M$ or $\{0,1\}^A$ ↳ state $\underline{\mathbf{x}} \in \mathbb{R}^N$ or $\{0,1\}^S$	select optimal action: $\underline{\mathbf{a}}^* = \underline{\mathbf{f}}(\underline{\mathbf{x}}, r)$ transition model: $P(\underline{\mathbf{x}}^{(t)} \mid \underline{\mathbf{x}}^{(t-1)}, \underline{\mathbf{a}}^{(t-1)})$

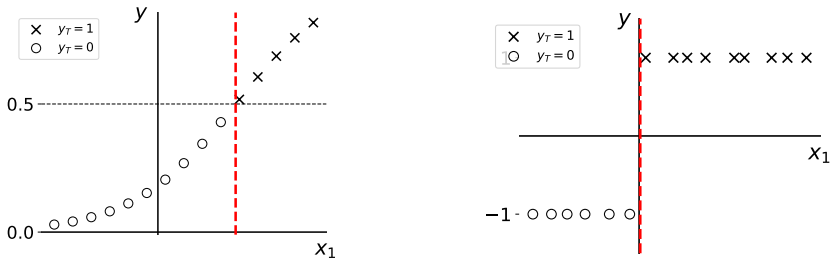
1.3 Connectionist Neuron (Perceptron)

A neuron is a computational unit for processing information.

Q 1.3.1: *What does a connectionist neuron compute?*

- A connectionist neuron is a type of neuron model which measures for a feature within an observation (a data point). It is a feature detector/extractor e.g. edge detector, above or below a threshold.

A connectionist neuron's response to 1D data:



(a) (b)

Figure 1.3.1: Examples of different neuron responses to scalar input of two types: $\times$ and $\circ$. In 1.3.1a: The neuron's response y is continuous. In 1.3.1b the neuron repsonse is $+1$ for positive input and -1 otherwise. The red lines act as a decision boundary.

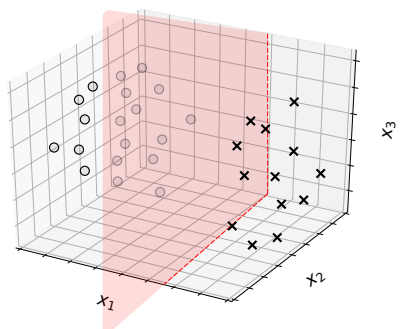


Figure 1.3.2: Examples of a neuron's response to 3D input of types $\times$ and $\circ$. Inputs of different types fall on opposite sides of a plane. The red plane acts as a decision boundary

1.3.1 Components of the connectionist neuron

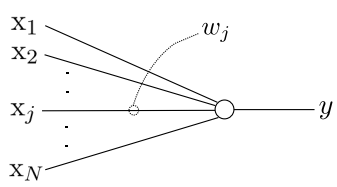


Figure 1.3.3: A graphical representation of the input-output relationship for a connectionist neuron. w_j describes the connection from the j-th input.

Fig. 1.3.3 is a diagram of a connectionist neuron. Given an input vector $\underline{\mathbf{x}} \in \mathbb{R}^N$ with components x_j, the connectionist neuron is composed of the following elements:

(a) weights $\underline{\mathbf{w}} \in \mathbb{R}^N$: The weights represent the strength of the connections between the neuron and each component of the input it receives.

(b) A linear filter: Summation of the weighted inputs, i.e. scalar product: $\underline{\mathbf{w}}^\top \underline{\mathbf{x}} = \sum_j w_j x_j$.

(c) A bias value $\theta \in \mathbb{R}$ also known as the *threshold* of the neuron.

(d) An activation function or transfer function $f : \mathbb{R} \mapsto \mathbb{R}$.

$f(\cdot)$ controls the range of the neuron's response. It can have the effect of squashing the response to a specific range of values or a specific set of values and preventing other value ranges.

(e) The scalar output of the neuron: y.

Q 1.3.2: *How does one interpret the scalar product $\underline{\mathbf{w}}^\top \underline{\mathbf{x}}$?*

The scalar product (dot product) acts as measure of similarity between $\underline{\mathbf{w}}$ and $\underline{\mathbf{x}}$. By looking at the geometric interpretation of the scalar product (dot product):

$$\underline{\mathbf{w}}^\top \underline{\mathbf{x}} = ||\underline{\mathbf{w}}||\ ||\underline{\mathbf{x}}||\ \cos\beta \tag{1.3.1}$$

where β is the angle between the two vectors. Therefore, $\underline{\mathbf{w}}^\top \underline{\mathbf{x}}$:

- is maximal when both vectors point in the same direction.
- It is zero if both vectors are orthogonal.
- It is maximally negative when both vectors point in opposite directions.

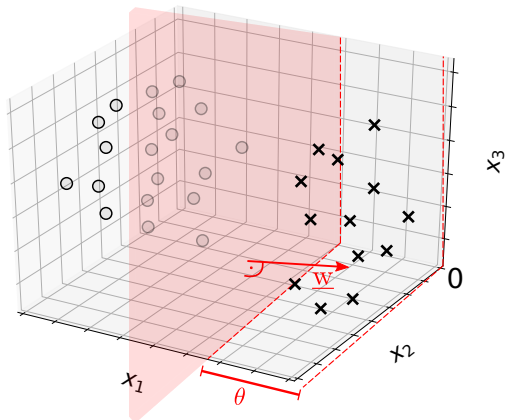


Figure 1.3.4: The plane divides the responses of the neuron. In the context of classification it is referred to as the decision boundary.

The input-output relationship is described by a linear filter with a static non-linearity $f(\cdot)$:

$$y = f\Big(\underbrace{\sum_{j=1}^{N} w_j x_j - \theta}_{=:h} \Big) = f\big(\underline{\mathbf{w}}^\top \underline{\mathbf{x}} - \theta \big) = f(h) \tag{1.3.2}$$

$\underline{\mathbf{x}}$:	input vector with components x_j
y :	scalar output of the neuron
$\underline{\mathbf{w}}$:	weight vector of the neuron with components w_j
θ :	threshold of the neuron
h :	total input of the neuron.
	$h \propto \frac{\mathbf{w}^\top \mathbf{x}}{\|\|\underline{\mathbf{w}}\|\|_2}$ component of $\underline{\mathbf{x}}$ in the direction of $\underline{\mathbf{w}}$
$f(\cdot)$:	transfer function

Q 1.3.3: *What roles do the weights and bias play?*

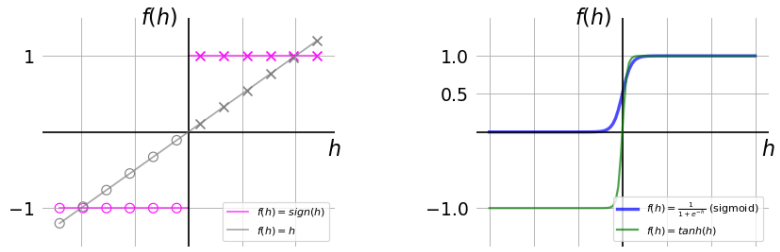
- $\underline{\mathbf{w}}$ is effectively the normal vector of the hyperplane $H = \{\underline{\mathbf{x}} : \underline{\mathbf{w}}^\top \underline{\mathbf{x}} = \theta\}$. Therefore, the weights represent the orientation of the hyperplane (see. Fig. 1.3.4).
- θ represents the shift of the hyperplane (see. Fig. 1.3.4). It is the absolute position of the hyperplane fron the origin along $\underline{\mathbf{w}}$. For any point $\underline{\widetilde{\mathbf{x}}} \in H$ (i.e. points on the plane): $\frac{\mathbf{w}^\top \underline{\widetilde{\mathbf{x}}}}{\|\|\underline{\mathbf{w}}\|\|_2} = \frac{\theta}{\|\|\underline{\mathbf{w}}\|\|_2}$

1.3.2 Linear vs. non-linear transfer functions

Q 1.3.4: *What are the advantages of using a non-linear transfer function instead of a linear one?*

- Advantages:

1. binary classification, either $f(h) \in \{0, 1\}$ or $f(h) \in (-1, 1)$

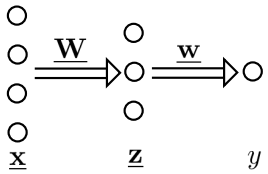


(a) Linear vs. non-linear activation

(b) logistic sigmoidal $f(h)=\frac{1}{1+e^{-h}}$ vs. tanh

Figure 1.3.5: Comparing linear with non-linear activation functions and differentiable alternatives to the sign function.

2. interpret $f(h)$ as a probability. The logistic sigmoidal where $f(h) = \frac{1}{1+exp(-h)}$ yields values in the range of (0,1). The logistic sigmoidal can also be obtained by shifting and scaling the tanh function (see Section 1.3.2.1.
3. a multilayer perceptron with only linear transfer functions can be reduced to a single layer. The hidden layers become redundant [1]:



$$\begin{aligned} y &= \underline{\mathbf{w}}^\top \underline{\mathbf{z}} = \underline{\mathbf{w}}^\top \underline{\mathbf{W}}\,\underline{\mathbf{x}} \\ &= (\underbrace{\underline{\mathbf{W}}^\top \underline{\mathbf{w}}}_{=:\underline{\widehat{\mathbf{w}}}})^\top \underline{\mathbf{x}} = \underline{\widehat{\mathbf{w}}}^\top \underline{\mathbf{x}} \\ &\widehat{=} \text{ connectionist neuron} \end{aligned}$$

1.3.2.1 From logistic sigmoidal to tanh

Q 1.3.5: *How does the logistic sigmoidal function relate to the tanh function?*

We observe in Fig. 1.3.5b that the logistic sigmoidal function (sigmoid) function is a shifted and scaled variant of the tanh function.

From this follows:

$$f_{\text{sigmoid}}(h) = \frac{1}{1+e^{-h}} \tag{1.3.3}$$

$$= \frac{e^{\frac{h}{2}}}{e^{\frac{h}{2}} + e^{-\frac{h}{2}}} \tag{1.3.4}$$

$$= \frac{1}{2}\left(\underbrace{\frac{e^{\frac{h}{2}} - e^{-\frac{h}{2}}}{e^{\frac{h}{2}} + e^{-\frac{h}{2}}}}_{=\tanh\frac{h}{2}} + \underbrace{\frac{e^{\frac{h}{2}} + e^{-\frac{h}{2}}}{e^{\frac{h}{2}} + e^{-\frac{h}{2}}}}_{=1}\right) \tag{1.3.5}$$

$$= \frac{1}{2}\left(\tanh\frac{h}{2} + 1\right) \tag{1.3.6}$$

[1] Recent literature reveals interesting insights into what kind of representations arise during learning across the layers of linear networks. For more information see (**?**).

1.3.3 Shortcut notation for weights and bias

The bias is effectively a connection between the neuron and an input that is always on. We can absorb the bias into the weight vector by prepending it to $\underline{\mathbf{w}}$ and prepending $\underline{\mathbf{x}}$ with an element $x_0 = 1$:

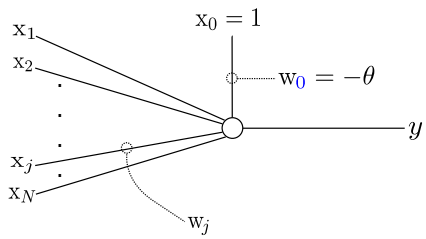


Figure 1.3.6: Absorb bias into weight vector.

The response of the neuron depicted in Fig. 1.3.6 can be computed by:

$$y = f\Big(\sum_{j=0}^{N} \mathrm{w}_j \mathrm{x}_j\Big) = f(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}) \tag{1.3.7}$$

Note that the sum in Eq. (1.3.7) now iterates from $j = 0$ instead of $j = 1$ as was done in Eq.(1.3.2) in order to include the bias element.

$\underline{\mathbf{w}}$ will be used for $\begin{pmatrix} \mathrm{w}_1 \\ \vdots \\ \mathrm{w}_N \end{pmatrix}$ as well as for $\begin{pmatrix} \mathrm{w}_0 \\ \mathrm{w}_1 \\ \vdots \\ \mathrm{w}_N \end{pmatrix}$.

Accordingly, $\underline{\mathbf{x}}$ will be used for $\begin{pmatrix} \mathrm{x}_1 \\ \vdots \\ \mathrm{x}_N \end{pmatrix}$ as well as for $\begin{pmatrix} \mathrm{x}_0 \\ \mathrm{x}_1 \\ \vdots \\ \mathrm{x}_N \end{pmatrix}$.

Whether the bias is absorbed or not should become apparent from the context or explicitly from the limits of the sum.

1.4 References

References

Andrew M Saxe, James L McClelland, and Surya Ganguli. A mathematical theory of semantic development in deep neural networks. *Proceedings of the National Academy of Sciences*, 116(23):11537–11546, 2019.